FunC v0.5.0
Ergonomic and Strongly Typed Design Proposal

Tomi Setsu

April 18, 2026

Abstract

This document proposes a new source-level design target for FunC, version `0.5.0`. It starts from the currently implemented FunC `0.4.6` surface documented in `doc/func.tex`, but reworks the language around five principles:

1. strong domain types for TVM/TOS values such as typed cells, `Slice`, `Builder`, `Address`, and typed messages;
2. algebraic data types and exhaustive pattern matching;
3. explicit fallibility through `Option` and `Result`;
4. explicit mutability and explicit side effects;
5. module-, trait-, and generic-oriented reuse suitable for a standard library and contract ecosystem.

The language direction intentionally moves toward Rust-like ergonomics, but does not attempt to become "Rust on TVM". In particular, it does not introduce ownership, borrowing, lifetimes, macro metaprogramming, or a synchronous call model that would conflict with TOS's asynchronous, actor-like execution.

This is a design proposal, not an implementation manifest. Its role is to define a coherent, teachable next-generation FunC surface that remains faithful to TVM/TOS semantics.

Introduction

Current FunC is compact and close to TVM, but that same closeness makes it hard to learn and easy to misuse. Several recurring pain points follow directly from the `0.4.6` language shape:

- domain values are not modeled as strongly as they should be;
- boolean logic, parsing success, and protocol errors are too often represented as unstructured integers;
- mutation is hidden inside special syntax such as `~method`;
- exceptions and implicit runtime conventions are used where typed return values would be easier to reason about;
- code reuse relies too heavily on low-level built-ins and ad hoc include patterns.

FunC `0.5.0` addresses these issues by keeping the low-level power of TVM available while moving ordinary authoring toward a safer and more legible surface. The intended result is not "higher level at any cost", but a better alignment between source code and the actual constraints of on-chain execution.

Contents

1 Status and Positioning

1.1 Status

This document is a **draft design proposal**. It defines the desired surface language, type system, and standard-library direction for a future FunC `0.5.0` edition. It does not claim that the current compiler already implements these features.

1.2 Relationship to FunC 0.4.6

FunC `0.4.6` is the implementation baseline. FunC `0.5.0` is the language-design target. The design aims to preserve the following invariants:

- compilation still targets TVM/Fift-style low-level code;
- TVM cells, slices, builders, continuations, and message actions remain first-class concerns;
- asynchronous cross-contract messaging remains the fundamental execution model;
- low-level escape hatches continue to exist for system code and stdlib authors.

1.3 What This Proposal Is Not

FunC `0.5.0` is *not* intended to be:

- a clone of Solidity;
- a full Rust compiler for TVM;
- a language that hides gas, bounce, serialization, or asynchronous delivery;
- a language that requires ownership and lifetime reasoning for ordinary contract code.

2 First Principles

2.1 Execution Model Comes First

The language must reflect the real execution model of TVM/TOS rather than a familiar but misleading model imported from EVM or general-purpose systems programming.

The relevant protocol facts are:

1. each contract owns isolated state;
2. cross-contract interaction happens through asynchronous messages;
3. a message may bounce, arrive later, or never arrive;
4. fees, reserve actions, and outgoing messages are explicit protocol operations;
5. serialization into cells is fundamental rather than incidental.

These facts impose direct source-language consequences:

1. contract-visible types must distinguish addresses, messages, coins, and raw cells;
2. fallibility should be part of the type system instead of being encoded as integer conventions;
3. mutation should be visible in the source rather than hidden in naming conventions;
4. side effects should be visible in signatures rather than collapsed into a single catch-all `impure` marker;
5. low-level TVM intrinsics should mostly live in the standard library, not in the parser.

2.2 Human Factors Matter

FunC `0.5.0` is optimized for the tasks that contract authors actually perform:

- define message schemas;
- parse inbound messages;
- update persistent state;
- send typed outbound messages;
- model recoverable failures clearly;
- reuse audited components without reading raw TVM stack code.

The language should therefore bias toward:

- readable declarations;
- local reasoning;
- exhaustiveness checks;
- effect visibility;
- error paths that are explicit and grep-able.

2.3 Non-Goals

To preserve approachability, FunC `0.5.0` deliberately excludes several features often associated with Rust:

- no ownership-and-lifetime borrow checker in the Rust sense;
- no lifetimes;
- no user-defined macros;
- no trait objects as a required abstraction mechanism;
- no `async/await`;
- no hidden heap allocation model.

At the same time, the language *may* include a lightweight checker for mutable-place overlap. Rejecting obviously dangerous expressions such as "mutate the same slice twice in one expression" is compatible with the goal of remaining easy to learn; it does not require authors to reason about lifetimes or ownership graphs.

3 Headline Changes with Respect to 0.4.6

Concern	**FunC 0.4.6**	**FunC 0.5.0 proposal**
Local binding	`var x = ...` or explicit type application syntax	`let x = ...`, `let mut x = ...`, and ordinary typed bindings
Boolean values	commonly encoded as `int`	dedicated `bool` type with no implicit int-to-bool coercion
Parsing failures	integer flags or exceptions	`Option<T>` and `Result<T, E>`
Mutating methods	`~method` syntax	ordinary method syntax with `mut self` in the signature
Error handling	`throw_*` helpers and `try/catch`	typed `Result` plus `?` and `match`; raw abort still exists for low-level code
Domain types	`int/cell/slice/builder/cont` plus conventions	built-in domain types such as `Cell<T>`, `Slice`, `Builder`, `Address`, `Coins`, and `InternalContext<T>`
Control over impurity	single `impure` qualifier	explicit effect sets such as `effect(read, write, send)`
Message schemas	manual op-code parsing and slice plumbing	`enum`, `struct`, codec traits, and typed entry-points

Serialization	manual builder/slice plumbing for every protocol type	`Cell<T>`, `to_cell`, `from_cell`, `load_any`, and `store_any` as ordinary typed APIs
Mutable alias safety	mostly unchecked beyond surface syntax	explicit mutable arguments plus a lightweight overlap checker for mutable places
Reuse	include-heavy and built-in-heavy	`mod`, `use`, generics, `trait`, and `impl`
Built-ins	many parser-predefined names	small core + stdlib `extern "tvm"` declarations

4 Design Goals

4.1 Primary Goals

FunC `0.5.0` has the following goals:

1. preserve TVM/TOS fidelity;
2. reduce incidental syntax complexity;
3. make domain concepts first-class in the type system;
4. make recoverable failure ordinary and typed;
5. make mutation and effects explicit;
6. make modules, traits, and generics the primary reuse mechanism;
7. keep low-level interop available for advanced code and compiler-adjacent libraries.

4.2 Compatibility Goals

The language should permit a staged migration from existing FunC contracts. Legacy code should not need to be rewritten in one step. Instead, the compiler should support an edition or compatibility mode in which old constructs are accepted while new constructs are gradually adopted.

5 Lexical Structure

5.1 Whitespace and Comments

FunC `0.5.0` adopts a more conventional comment system:

- `//` introduces a line comment;
- `/* ... */` introduces a nested block comment.

For migration purposes, the legacy FunC comment forms `;;` and `{- ... -}` may remain accepted in compatibility mode, but they are not part of the preferred `0.5.0` surface.

5.2 Identifiers

Ordinary identifiers use a conventional snake-case-oriented shape:

```
[A-Za-z_][A-Za-z0-9_]*
```

Module-qualified names use `::`

```
std::slice::SliceReader
wallet::msg::Transfer
```

Predicate names should use `is_*` or `has_*` rather than the old `name?` convention. Fallible operations should use `try_*` or return `Result`.

5.3 Literals

The preferred literal forms are:

- integers: `0`, `17`, `0xff`, `42u32`, `-1i257`;
- booleans: `true`, `false`;
- strings: `"hello"`;
- byte strings: `b"abc"`;
- hexadecimal byte strings: `hex"deadbeef"`;
- addresses: `addr"EQC..."` or `addr"-1:abcd..."`.

The old string-suffix forms from FunC `0.4.6` may remain available behind compatibility gates, but new code should prefer explicit prefixes and typed constructors.

5.4 Keywords

The core language reserves the following new keyword set:

```
mod use pub const type struct enum message trait impl fn extern
let mut static match if else while loop for in break continue
return where self Self as
effect pure unsafe
entry external internal bounce get
```

Compatibility mode may additionally reserve legacy FunC words such as `global`, `asm`, `impure`, `inline`, and `method_id`.

6 Source Structure and Modules

6.1 Compilation Unit

A source file is a sequence of items:

```
Item ::= ModuleDecl
       | UseDecl
       | ConstDecl
       | TypeAlias
       | StructDecl
       | EnumDecl
       | MessageDecl
       | TraitDecl
       | ImplDecl
       | FunctionDecl
```

6.2 Modules

Modules organize contract code and the standard library:

```
mod wallet::state;
mod wallet::msg;
use std::result::Result;
use std::message::{InternalContext, SendOptions};
```

The language keeps modules simple:

- there is no macro expansion model tied to modules;
- visibility is controlled by `pub`;
- module-qualified names use `::`;
- `use` imports names into local scope.

6.3 Public API Surface

Every item is private to its module unless marked `pub`. This matters for standard library design, contract libraries, and test-only helpers:

```
pub struct WalletState { ... }
pub enum WalletMsg { ... }
pub message Transfer(op = 0x0f8a7ea5) { ... }
pub fn recv_internal(...) -> Result<(), WalletError> effect(write, send) { ... }
```

7 Type System

7.1 Built-In Scalar Types

The proposed scalar types are:

```
bool
i32  i64  i128  i256  i257
u8   u16  u32  u64  u128  u256
```

`i257` remains available because TVM integers are fundamentally 257-bit signed values. For migration, `int` may remain an alias of `i257` in compatibility mode, but new code should prefer explicit widths.

7.2 Built-In Domain Types

The following types become first-class built-ins:

```
Cell<T>
Slice
Builder
Cont
Address
StdAddress
Coins
Bytes
RawCell
RawSlice
```

The role of these types is:

- `Cell<T>` represents a typed cell reference carrying an expected decoded payload type;
- `Slice` and `Builder` model the TVM read/write cell domain;
- `Address` models a full on-chain address, not an arbitrary slice;
- `Coins` models currency amounts and documents intent better than raw `u128`;

- `Bytes` represents ordinary byte strings;
- `RawCell` and `RawSlice` are low-level escape-hatch types for unchecked code.

7.3 Typed Cell References

Typed cells are central to making TVM serialization tractable without hiding it:

```
pub struct WalletState {
    seqno: u32,
    owner: Address,
    plugins: Cell<PluginDict>?,
}
```

`Cell<T>` is assignable to `RawCell` when raw interop is needed, but ordinary contract code should prefer the typed form. This improves reviewability for persistent state, lazy-loaded message refs, and nested TL-B objects.

7.4 Compound Types

Compound types are:

```
(T1, T2, ..., Tn)          // tuples
[T; N]                     // fixed-size arrays
T?                         // sugar for Option<T>
Option<T>
Result<T, E>
```

Tuples remain useful because TVM naturally works with multiple stack results, but user-defined `struct` and `enum` become the preferred way to describe meaningful protocol data.

7.5 Struct Types

Structures are declared as:

```
pub struct WalletState {
    seqno: u32,
    public_key: u256,
    plugins: Dict<PluginKey, ()>,
}
```

Struct fields are named, typed, and immutable by default. Mutation happens through a mutable binding or a method with `mut self`.

7.6 Enum Types

Enumerations are algebraic data types and are central to the proposal:

```
pub enum WalletMsg {
    Transfer {
        query_id: u64,
        amount: Coins,
        to: Address,
        response_to: Option<Address>,
    },
    InstallPlugin {
        plugin: Address,
    },
    RemovePlugin {
        plugin: Address,
    },
}
```

Enums may also carry explicit discriminants when required by wire formats:

```
pub enum Op : u32 {
    Transfer = 0x0f8a7ea5,
    Burn     = 0x595f07bc,
}
```

7.7 Message Declarations

Wire-level message bodies deserve a dedicated declaration form because they are one of the dominant concepts in TVM/TOS contracts:

```
pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
    response_to: Address?,
}
```

A `message` declaration behaves like a named struct plus a derived message codec. In particular:

- the opcode binding is part of the declaration rather than an external convention;
- the declaration derives the relevant cell/TL-B codec traits;
- typed entrypoints may request `InternalContext<Transfer>` directly;
- higher-level enums may still group multiple message bodies when a contract wants one sum type for dispatch.

7.8 Option and Result

Two standard prelude types are assumed:

```
pub enum Option<T> {
    Some(T),
    None,
}

pub enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

These types are not library afterthoughts. They are core to ordinary contract authoring:

- `Option<T>` expresses absence without sentinels;
- `Result<T, E>` expresses recoverable failure without bare exceptions;
- `match` and `?` make both types ergonomic in routine code.

For readability, `T?` is accepted as surface sugar for `Option<T>`:

```
Address?
Cell<JettonWalletState>?
```

Unlike languages that normalize everything around `null`, the teaching path still centers `Some`/`None`, `match`, and `if let`.

7.9 Type Aliases

Type aliases remain important for readability:

```
pub type QueryId = u64;
pub type PluginKey = (i32, u256);
pub type Ton = Coins;
```

7.10 Generics

Generic functions, structs, enums, and traits use angle-bracket syntax:

```
pub struct Dict<K, V> { ... }

pub fn first<T>(items: (T, T)) -> T { ... }
```

Unlike Rust, the design does not introduce lifetime parameters or ownership bounds. Generic programming is deliberately restricted to type parameters and trait constraints.

8 Traits and Implementations

8.1 Trait Declarations

Traits provide interface-oriented reuse:

```
pub trait TlbCodec {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure;
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure;
}
```

8.2 Implementations

Implementations attach methods to types:

```
impl WalletState {
    pub fn increment_seqno(mut self) -> Self pure {
        self.seqno += 1;
        self
    }
}

impl TlbCodec for WalletMsg {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure { ... }
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure { ... }
}
```

8.3 Why Traits Instead of Parser Magic

One design goal of `0.5.0` is to move functionality out of the parser and into audited, inspectable library code. Traits support that directly:

- codecs can be trait-based;
- message sending can depend on a trait bound rather than raw cells;
- collection and builder helpers can live in the standard library;
- test doubles and mock implementations become natural.

This does *not* mean the compiler should be blind to protocol structure. A small amount of protocol-aware lowering remains justified for things such as entrypoint contexts, message opcodes, and automatic inline-vs-ref message-body placement. The goal is to eliminate accidental parser magic, not to forbid every useful built-in lowering.

9 Bindings, Mutability, and Patterns

9.1 Immutable by Default

All local bindings are immutable unless declared with `mut`:

```
let seqno: u32 = state.seqno;
let mut cs: Slice = body;
```

This replaces the old mixture of `var`, explicit type application, and hidden mutation through `~method` calls.

9.2 Assignments

Only mutable places may be assigned to:

```
let mut state = load_state()?;
state.seqno = state.seqno + 1;
```

9.3 Pattern-Oriented Control Flow

Patterns are not limited to `match`. The language should also support `if let` and `while let` because optional values and typed message variants are ubiquitous in contract code:

```
if let Some(response_to) = msg.response_to {
    send(response_to, Ack { query_id: msg.query_id }, opts)?;
}

while let Some(item) = queue.pop_front() {
    process(item)?;
}
```

9.4 Destructuring

Patterns work in `let` bindings and in `match`:

```
let (amount, response_to) = (msg.amount, msg.response_to);
let WalletMsg::Transfer { amount, to, .. } = msg;
```

Pattern forms include:

- wildcard `_`;
- name bindings;
- tuple patterns;
- struct patterns;
- enum variant patterns.

9.5 Mutating Methods

Methods may declare `mut self`:

```
impl Slice {
    pub fn load_uint(mut self, bits: u16) -> Result<u256, ParseError> pure { ... }
}
```

At the call site, this looks ordinary:

```
let mut cs = body;
let op = cs.load_uint(32)?;
```

This replaces the special `cs~load_uint(32)` syntax. The source clearly states mutability in the binding and in the method signature, rather than hiding it in punctuation.

9.6 Mutable Arguments and Lightweight Safety Checks

Methods with `mut self` remain the idiomatic surface for builder/slice-like APIs. Free functions may also accept mutable places explicitly:

```
pub fn load_var_uint(mut s: Slice, bits: u16) -> Result<u256, ParseError> pure { .

let mut cs = body;
let n = load_var_uint(mut cs, 5)?;
```

This proposal adopts three lightweight rules:

- only mutable lvalues may be passed as `mut` arguments;
- the same mutable place may not be passed twice in one full expression;
- parent and child places may not be mutated simultaneously in one full expression.

These rules are intentionally narrow. They prevent obvious aliasing hazards without introducing ownership, region inference, or lifetime syntax.

10 Functions and Effect System

10.1 Function Syntax

The general form is:

```
[pub] fn name(arg1: T1, mut arg2: T2, ...) -> R [effect(...)] Block
```

Examples:

```
pub fn min(x: i257, y: i257) -> i257 pure { ... }
pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }
pub fn load_uint(mut s: Slice, bits: u16) -> Result<u256, ParseError> pure { ... }
pub fn send_transfer(...) -> Result<(), SendError> effect(send) { ... }
```

10.2 Pure by Default

A function with no effect annotation is `pure`. It may compute, match, and transform values, but it may not:

- read persistent state or chain context;
- modify persistent state;
- emit outgoing messages;
- use raw unsafe TVM intrinsics.

10.3 Effect Sets

The primary effect vocabulary is:

```
effect(read)
effect(write)
effect(send)
effect(unsafe)
```

These effects mean:

- `read`: may inspect persistent state, inbound context, config, time, balance, or other chain environment values;
- `write`: may persist updated state or otherwise mutate contract-owned storage;
- `send`: may emit action-phase effects such as outbound messages, reserve operations, or code changes;
- `unsafe`: may use raw assembler, unchecked parsing, or other operations whose safety cannot be proved from the surface type rules alone.

An effect set is cumulative: `effect(write, send)` may both write state and send messages.

10.4 Why Not Keep `impure`

The old `impure` marker is too coarse. It does not distinguish a getter that reads block context from a function that mutates storage and sends three messages. In `0.5.0`, effect sets become part of ordinary interface review.

10.5 Entrypoints

Contract-facing entrypoints are explicit:

```
entry external fn recv_external(
    ctx: ExternalContext<SignedRequest>,
) -> Result<(), WalletError> effect(read, write, send) { ... }

entry internal fn recv_internal(
    ctx: InternalContext<WalletMsg>,
) -> Result<(), WalletError> effect(read, write, send) { ... }

entry bounce fn on_bounce(
    ctx: BounceContext<RawSlice>,
) -> Result<(), WalletError> effect(read, write) { ... }
```

Getter functions use a dedicated modifier:

```
get fn seqno() -> Result<u32, ParseError> effect(read) { ... }
```

The compiler maps these to the existing TVM/TOS conventions for internal, external, bounced, and get-method execution.

The context types are not ordinary heap objects. They are compiler-recognized views over real VM inputs. Source code should nevertheless treat them as normal, typed fields:

```
ctx.sender
ctx.value
ctx.forward_fee
ctx.body
```

This makes the execution model explicit while keeping raw TVM stack plumbing out of routine contract code.

10.6 External Intrinsics

Raw TVM primitives should be declared as `extern "tvm"` in the standard library rather than being magic parser names:

```
extern "tvm" fn send_raw_message(msg: RawCell, mode: u8) -> () effect(send, unsafe
extern "tvm" fn accept_message() -> () effect(send, unsafe);
```

This preserves power while making the surface language smaller and more teachable.

11 Expressions and Statements

11.1 Core Statements

The language supports:

```
let
let mut
if / else
if let
match
while
while let
loop
for
break
continue
return
```

11.2 Match Expressions

Pattern matching is exhaustive by default:

```
match msg {
    WalletMsg::Transfer { query_id, amount, to, response_to } => { ... }
    WalletMsg::InstallPlugin { plugin } => { ... }
    WalletMsg::RemovePlugin { plugin } => { ... }
}
```

This is a major ergonomics improvement over manually parsing an op code into an `int` and then maintaining a parallel maze of `if` branches.

11.3 Flow-Sensitive Narrowing

Pattern matching and optional-value tests should refine types in a flow-sensitive way:

```
if let Some(reply_to) = msg.response_to {
    send(reply_to, Receipt { query_id: msg.query_id }, opts)?;
}
```

Inside the `if let` body, `reply_to` is an `Address` rather than `Address?`. Similar narrowing should happen for `match` arms and other obvious success paths over `Option`, `Result`, and message variants.

11.4 The Question-Mark Operator

If a function returns `Result<T, E>`, the `?` operator may be applied to intermediate `Result` values:

```
let mut cs = body;
let global_id = cs.load_int(32)?;
let seqno = cs.load_uint(32)?;
let msg = WalletMsg::load(cs)?;
```

This drastically reduces the need for hand-written `throw_unless`, `try/catch`, and integer-status plumbing in ordinary code.

11.5 Loop Design

The preferred loop forms are:

```
while cond { ... }
loop { ... }
for item in items { ... }
```

Legacy `repeat` and `do ... until` may remain in compatibility mode, but the core language should converge on a smaller, more conventional control surface.

12 Messages, Serialization, and TVM-Specific APIs

12.1 Typed Message Contexts

Messages should be represented as typed values rather than raw slices whenever possible. The standard library is expected to define context types such as:

```
pub struct InternalContext<T> {
    sender: Address,
    value: Coins,
    forward_fee: Coins,
    body: T,
}

pub struct ExternalContext<T> {
    body: T,
    signature: Option<Bytes>,
}

pub struct BounceContext<T = RawSlice> {
    sender: Address,
    value: Coins,
    original_forward_fee: Coins,
    body: T,
}
```

When an entrypoint requests `InternalContext<T>` for a concrete message type, the compiler or stdlib-generated wrapper is responsible for parsing the inbound body into `T`. Contracts may still opt into `RawSlice` when they need manual parsing.

12.2 Typed Cell APIs

Serialization should be both trait-driven and convenient in routine code:

```
impl<T: CellEncode> T {
```

```
    pub fn to_cell(self) -> Result<Cell<T>, CellError> pure;
}

impl<T: CellDecode> Cell<T> {
    pub fn load(self) -> Result<T, ParseError> pure;
    pub fn begin_parse(self) -> Slice pure;
}
```

12.3 Codec Traits

Serialization and deserialization should be expressed through traits:

```
pub trait CellEncode {
    fn encode(self) -> Result<RawCell, CellError> pure;
}

pub trait CellDecode: Sized {
    fn decode(cell: RawCell) -> Result<Self, ParseError> pure;
}
```

For wire protocols that rely on TL-B, a narrower `TlbCodec` trait may specialize the same idea.

The standard library should also offer high-frequency helpers modeled directly on how contracts are written:

```
impl Slice {
    pub fn load_any<T: CellDecode>(mut self) -> Result<T, ParseError> pure;
}

impl Builder {
    pub fn store_any<T: CellEncode>(mut self, value: T) -> Result<Self, CellError>
}
```

12.4 Slice and Builder APIs

The low-level APIs remain essential, but their contracts become explicit:

```
impl Slice {
```

```
    pub fn load_uint(mut self, bits: u16) -> Result<u256, ParseError> pure;
    pub fn load_int(mut self, bits: u16) -> Result<i257, ParseError> pure;
    pub fn load_ref(mut self) -> Result<RawCell, ParseError> pure;
    pub fn load_addr(mut self) -> Result<Address, ParseError> pure;
    pub fn remaining_bits(self) -> u16 pure;
    pub fn remaining_refs(self) -> u8 pure;
}

impl Builder {
    pub fn store_uint(mut self, value: u256, bits: u16) -> Result<Self, CellError>
    pub fn store_int(mut self, value: i257, bits: u16) -> Result<Self, CellError>
    pub fn store_ref(mut self, cell: RawCell) -> Result<Self, CellError> pure;
    pub fn end_cell(self) -> Result<RawCell, CellError> pure;
}
```

Note that parsing and building remain explicit operations. The language does not try to make TVM cells disappear; it makes them typed and tractable.

12.5 Sending Messages

High-level message sending should take typed bodies:

```
pub enum BounceMode {
    NoBounce,
    Body256,
    Rich,
}

pub enum BodyLayout {
    Auto,
    Inline,
    ByRef,
}

pub struct SendOptions {
    value: Coins,
    bounce: BounceMode,
```

```
    body_layout: BodyLayout,
    mode: SendMode,
}

pub fn send<T: CellEncode>(
    dst: Address,
    body: T,
    opts: SendOptions,
) -> Result<(), SendError> effect(send);
```

This API expresses a real protocol operation more directly than ad hoc raw-cell construction plus `SENDRAWMSG`.

When `body_layout` is `Auto`, the compiler or stdlib may inline small message bodies and spill larger ones to a ref automatically. This reflects real TVM ergonomics: source code should express intent, while the message builder chooses the cheapest valid encoding unless the author overrides it.

13 Error Model

13.1 Recoverable and Unrecoverable Failure

FunC `0.5.0` distinguishes:

- recoverable failures, represented by `Result<T, E>`;
- unrecoverable VM aborts, represented by `abort` or raw unsafe intrinsics.

The default teaching path should strongly prefer typed recoverable failure.

13.2 Domain Error Types

Contracts are expected to define domain-specific error enums:

```
pub enum WalletError {
    InvalidSignature,
    WrongGlobalId,
    WrongSeqno,
```

```
    UnknownPlugin,
    Parse(ParseError),
    Send(SendError),
}
```

13.3 Bridging Legacy Exit Codes

Compatibility layers may map legacy numeric exit codes to domain enums:

```
pub enum LegacyExitCode : u16 {
    InvalidSignature = 202,
    WrongSeqno = 33,
}
```

This lets modern source code remain typed even when it interoperates with legacy tooling or protocol conventions.

14 Standard Library Direction

14.1 Small Core, Rich Stdlib

The language core should stay small. Most functionality that is currently special or parser-defined should instead move to the standard library:

- arithmetic helper functions;
- slice and builder utilities;
- dictionary abstractions;
- message and context types;
- codec traits and implementations;
- contract-state helpers;
- cryptographic intrinsics surfaced as `extern "tvm"` functions.

Expected stdlib conveniences include:

- `create_message` and `send` wrappers over raw action lists;

- typed state load/save helpers built on `Cell<T>`;
- `load_any` / `store_any` helpers for routine TL-B work;
- message-body declarations with derived codecs.

14.2 Traits as the Unit of Reuse

Traits provide the right level of abstraction for:

- codecs;
- formatting and debugging;
- message sending;
- dictionary key/value constraints;
- reusable protocol behaviors.

This gives TOS a realistic path toward an audited contract standard library without forcing all composition through includes and naming conventions.

15 What Remains Explicit

The proposal intentionally keeps several low-level concerns visible:

- serialization into cells;
- bounce handling;
- coins attached to outgoing messages;
- effectful operations such as `send` and reserve actions;
- unsafe raw TVM interop where needed.

This is a language for TVM/TOS, not an attempt to hide the platform.

16 Compatibility and Migration

16.1 Edition Model

The recommended migration path is edition-based. A source file may opt into the new syntax and semantics explicitly:

```
#![edition = "0.5.0"]
```

Legacy files may continue to compile under a compatibility edition:

```
#![edition = "0.4-compat"]
```

16.2 Compatibility Surface

The compatibility edition may continue to accept:

- legacy type names such as `int` and `cell`;
- legacy comments;
- legacy `global`, `const`, `asm`, `impure`, `inline`, `method_id` forms;
- legacy `~method` mutation syntax;
- legacy exception-style helpers.

16.3 Illustrative Source Migration

```
// 0.4.6 style
var cs = in_msg;
var signature = in_msg~load_bits(512);
throw_unless(33, msg_seqno == stored_seqno);

// 0.5.0 style
let mut cs = in_msg;
let signature = cs.load_bits(512)?;
if msg_seqno != stored_seqno {
    return Err(WalletError::WrongSeqno);
}
```

16.4 Migration Principle

The important migration principle is semantic, not cosmetic:

- move from raw integers toward explicit types;
- move from hidden mutation toward `mut`;
- move from integer-status parsing toward `Result`;
- move from raw op-code dispatch toward `enum + match`;
- move from parser magic toward stdlib declarations, while retaining a small amount of protocol-aware lowering where it clearly improves source-level clarity.

17 Worked Comparison: One Wallet Contract in 0.4.6 and 0.5.0

The following comparison shows the same piece of contract logic in both language styles:

1. load wallet state;
2. parse an inbound transfer request;
3. validate the transfer sequence number;
4. send native coins to the destination;
5. increment and persist the sequence number.

The examples are intentionally compact. They are not meant to be full production wallets; they are meant to expose the source-language shape of the same task.

17.1 FunC 0.4.6 Style

In current-style FunC, the code is dominated by raw slice parsing, implicit mutation through `~method` syntax, integer-coded failures, and manual message construction:

```
;; FunC 0.4.6 style

(int, slice) load_data() inline {
  var ds = get_data().begin_parse();
  return (ds~load_uint(32), ds~load_msg_addr());
}

() save_data(int seqno, slice owner) impure inline {
  set_data(
    begin_cell()
      .store_uint(seqno, 32)
      .store_slice(owner)
      .end_cell()
  );
}

() recv_internal(slice in_msg_body) impure {
  var (stored_seqno, owner) = load_data();
  var cs = in_msg_body;

  int op = cs~load_uint(32);
  throw_unless(40, op == 0x0f8a7ea5);

  int msg_seqno = cs~load_uint(32);
  slice to = cs~load_msg_addr();
  int amount = cs~load_tomis();

  throw_unless(33, msg_seqno == stored_seqno);
  throw_unless(34, amount > 0);

  send_raw_message(
    begin_cell()
      .store_uint(0x18, 6)
```

```
      .store_slice(to)
      .store_tomis(amount)
      .store_uint(0, 1 + 4 + 4 + 64 + 32 + 1 + 1)
      .end_cell(),
    1
  );

  save_data(stored_seqno + 1, owner);
}
```

17.2 FunC 0.5.0 Style

In the proposed `0.5.0` surface, the same logic is expressed through typed state, a typed message declaration, explicit mutability, explicit effects, and a `Result`-based error path:

```
#![edition = "0.5.0"]

pub struct WalletState {
    seqno: u32,
    owner: Address,
}

pub message Transfer(op = 0x0f8a7ea5) {
    seqno: u32,
    to: Address,
    amount: Coins,
}

pub enum WalletError {
    WrongSeqno,
    ZeroAmount,
    Parse(ParseError),
    Send(SendError),
}

pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }
pub fn save_state(state: WalletState) -> Result<(), CellError> effect(write) { ...
```

```
entry internal fn recv_internal(
    ctx: InternalContext<Transfer>,
) -> Result<(), WalletError> effect(read, write, send) {
    let mut state = load_state()?;
    let transfer = ctx.body;

    if transfer.seqno != state.seqno {
        return Err(WalletError::WrongSeqno);
    }
    if transfer.amount.is_zero() {
        return Err(WalletError::ZeroAmount);
    }

    send(
        transfer.to,
        (),
        SendOptions {
            value: transfer.amount,
            bounce: BounceMode::Body256,
            body_layout: BodyLayout::Auto,
            mode: SendMode::PayFeesSeparately,
        }
    )?;

    state.seqno += 1;
    save_state(state)?;
    Ok(())
}
```

17.3 What the Comparison Shows

The comparison illustrates the intended source-level improvements:

- **Typed message declaration instead of raw op-code dispatch.** In `0.4.6` the contract manually loads `op`, then loads each field from a `slice`. In `0.5.0` the `Transfer` message binds its opcode in the declaration and the entrypoint receives a typed body directly.

- **Explicit domain types.** `Address` and `Coins` replace raw `slice` and `int` conventions, making the meaning of each field visible at the type level.
- **Explicit mutation.** In `0.4.6` mutation is hidden inside `cs~load_uint(...)` and similar forms. In `0.5.0` mutation is carried by `let mut cs` or `let mut state` and by explicit mutable parameters.
- **Typed recoverable errors instead of numeric abort codes.** `throw_unless(33, ...)` and `throw_unless(34, ...)` become `Err(WalletError::WrongSeqno)` and `Err(WalletError::ZeroAmount)`. This makes reviews and refactors much easier.
- **High-level send API instead of manual action-cell assembly.** The `0.4.6` version has to build an outbound message cell explicitly. The `0.5.0` version exposes the protocol operation as `send(...)`, with value, bounce mode, body layout, and mode still explicit.
- **Persistent state becomes a typed struct.** The state layout is no longer "a sequence of manually loaded fields" from the reader's point of view. The source speaks in terms of `WalletState`.

17.4 Why This Example Matters

This example is deliberately small, but the same pattern scales to larger contracts:

- Jetton and NFT messages become enums instead of raw op-code trees;
- single message bodies can be declared with bound opcodes instead of parallel "struct + opcode constant" conventions;
- parser helpers return `Result` instead of integer flags;
- bounce handlers match typed variants instead of re-parsing raw slices;
- audited stdlib modules provide `send`, codecs, and dict abstractions instead of every contract open-coding them.

18 Core Grammar Summary

The grammar below is intentionally schematic. It describes the shape of the language, not every precedence rule or parsing ambiguity.

```
Item ::= ModuleDecl
       | UseDecl
       | ConstDecl
       | TypeAlias
       | StructDecl
       | EnumDecl
       | MessageDecl
       | TraitDecl
       | ImplDecl
       | FunctionDecl

ModuleDecl ::= "mod" Path ";"
UseDecl    ::= "use" Path [ "as" identifier ] ";"
ConstDecl  ::= [ "pub" ] "const" identifier ":" Type "=" Expr ";"
TypeAlias  ::= [ "pub" ] "type" identifier [ GenericParams ] "=" Type ";"

StructDecl ::= [ "pub" ] "struct" identifier [ GenericParams ] "{" Fields "}"
EnumDecl   ::= [ "pub" ] "enum" identifier [ GenericParams ] [ ":" Type ] "{"
               Variants "}"
MessageDecl ::= [ "pub" ] "message" identifier [ "(" "op" "=" Expr ")" ]
                "{" Fields "}"
TraitDecl  ::= [ "pub" ] "trait" identifier [ GenericParams ] "{"
               TraitItems "}"
ImplDecl   ::= "impl" [ GenericParams ] Type [ "for" Type ] "{"
               ImplItems "}"

FunctionDecl ::= [ "pub" ] [ EntryQualifier ] "fn" identifier
                 [ GenericParams ] "(" Params ")" "->" Type
                 [ EffectQualifier ] BlockOrSemicolon

EntryQualifier ::= "entry" ( "external" | "internal" | "bounce" )
                 | "get"
```

```
EffectQualifier ::= "pure"
                  | "unsafe"
                  | "effect" "(" EffectList ")"

Stmt ::= LetStmt
       | ExprStmt
       | ReturnStmt
       | IfStmt
       | IfLetStmt
       | MatchStmt
       | WhileStmt
       | WhileLetStmt
       | LoopStmt
       | ForStmt
       | BreakStmt
       | ContinueStmt
       | Block

Type ::= SimpleType
       | Type "?"

Arg ::= Expr
      | "mut" Expr
```

19 Illustrative Contract Fragment

The following fragment demonstrates the overall style target:

```
#![edition = "0.5.0"]

use std::result::Result;
use std::message::{InternalContext, SendOptions, SendMode, send};

pub struct WalletState {
    seqno: u32,
    owner: Address,
}
```

```
pub enum WalletError {
    WrongSeqno,
    Parse(ParseError),
    Send(SendError),
}

pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
    response_to: Address?,
}

pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }
pub fn save_state(state: WalletState) -> Result<(), CellError> effect(write) { ...

entry internal fn recv_internal(
    ctx: InternalContext<Transfer>,
) -> Result<(), WalletError> effect(read, write, send) {
    let mut state = load_state()?;
    let msg = ctx.body;

    if let Some(response_to) = msg.response_to {
        send(
            response_to,
            Receipt { query_id: msg.query_id },
            SendOptions {
                value: 0.into(),
                bounce: BounceMode::NoBounce,
                body_layout: BodyLayout::Auto,
                mode: SendMode::PayFeesSeparately,
            },
        )?;
    }

    send(
        msg.to,
```

```
        msg,
        SendOptions {
            value: msg.amount,
            bounce: BounceMode::Body256,
            body_layout: BodyLayout::Auto,
            mode: SendMode::PayFeesSeparately,
        },
    )?;

    state.seqno = state.seqno + 1;
    save_state(state)?;
    Ok(())
}
```

20 Conclusion

FunC `0.5.0` should become easier to read not by hiding TVM/TOS, but by modeling it honestly:

- typed messages instead of raw slices;
- enums and exhaustive `match` instead of integer dispatch;
- message declarations with bound opcodes instead of ad hoc constants;
- typed cell references and auto codec helpers instead of repetitive builder/slice plumbing;
- `Result` and `Option` instead of status flags and surprise exceptions;
- `let mut` and `mut self` instead of punctuation-based mutation;
- `if let` and flow-sensitive narrowing for common optional/message control flow;
- explicit effects instead of a single opaque impurity bit;
- trait- and module-based reuse instead of parser magic and ad hoc includes.

That combination preserves the strengths of FunC — closeness to TVM, direct control over serialization and messaging, and efficient compilation — while making the source language much easier to teach, review, and scale across a real contract ecosystem.